AWS VPC PEERING

Private Connectivity Between Two Amazon Virtual Private Clouds

Student name	_____
Batch / course	_____
AWS Region	_____
Completion date	_____
Repository URL	Add after publishing to GitHub

Implementation status: report template prepared; live AWS configuration, measured test results, screenshots, and resource IDs must be filled in after you complete the lab.

1. Abstract

This project demonstrates private IPv4 communication between two Amazon Virtual Private Clouds (VPCs) using a VPC peering connection. VPC A contains a public bastion host and a private application instance. VPC B contains a second private application instance and has no Internet Gateway. Route tables and security groups control the traffic paths. The project validates baseline failure, bidirectional connectivity, SSH access through the bastion, and a controlled security-rule failure followed by recovery.

2. Objectives

• Design two VPCs with non-overlapping IPv4 CIDR blocks.

• Create public and private subnets and explicitly associate route tables.

• Establish and accept a VPC peering connection.

• Configure forward and return routes on the relevant route tables.

• Restrict inbound SSH and ICMP traffic using security groups.

• Test connectivity and document real evidence.

• Clean up resources to reduce ongoing AWS charges.

3. Architecture and IP addressing

VPC A uses 10.10.0.0/16 and VPC B uses 10.20.0.0/16. Their CIDRs do not overlap, which is required for this peering design.

Resource	Subnet / CIDR	Address / notes
ewb-vpc-a	VPC	10.10.0.0/16
ewb-a-public	Public subnet A	10.10.1.0/24
ewb-a-private	Private subnet A	10.10.2.0/24
ewb-bastion	Public subnet A	10.10.1.10 + public IPv4
ewb-app-a	Private subnet A	10.10.2.10; no public IP
ewb-vpc-b	VPC	10.20.0.0/16
ewb-b-private	Private subnet B	10.20.1.0/24
ewb-app-b	Private subnet B	10.20.1.10; no public IP

4. Route table design

Route table	Required routes	Purpose
ewb-rt-a-public	10.10.0.0/16 → local; 0.0.0.0/0 → Internet Gateway; 10.20.0.0/16 → peering	Internet access for bastion; route to VPC B
ewb-rt-a-private	10.10.0.0/16 → local; 10.20.0.0/16 → peering	App-A to VPC B
ewb-rt-b-private	10.20.0.0/16 → local; 10.10.0.0/16 → peering	Return path from VPC B

5. Security configuration

Bastion inbound SSH (TCP 22) should be allowed only from the student's current public IPv4 address using a /32 source. App-A and App-B should allow SSH only from the bastion private address 10.10.1.10/32. ICMP Echo Request should be limited to the opposite private application host's /32 address. Do not expose SSH or ICMP to 0.0.0.0/0. The

private application instances must not have public IPv4 addresses.

6. Implementation procedure

• Create VPC A and VPC B with the CIDRs above; create the specified subnets.

• Attach an Internet Gateway to VPC A and configure the public route table. Keep private subnets without an internet default route.

• Create and associate the private route tables and restricted security groups.

• Launch three Amazon Linux 2023 EC2 instances. Assign the planned private IPs; give a public IPv4 address only to the bastion.

• Create the VPC peering request from VPC A to VPC B and accept it. Confirm status is Active.

• Add the peering routes to the public A, private A, and private B route tables.

• Configure SSH ProxyJump locally, run baseline and post-configuration tests, and save screenshots.

• Complete cleanup after collecting evidence.

7. Test plan and results

Fill this table using your own AWS terminal output and screenshots. The expected outcomes below are acceptance criteria from the project design, not results already observed.

Test	Command / evidence	Expected result	Actual result
Baseline	App-A: ping -c 4 10.20.1.10 before peering/routes	No replies	Fill after test
Forward path	App-A: ping -c 4 10.20.1.10 after routes	4 replies; 0% packet loss	Fill after test
Reverse path	App-B: ping -c 4 10.10.2.10	4 replies; 0% packet loss	Fill after test
SSH via bastion	Laptop: ssh ewb-app-b	Shell on App-B	Fill after test
Isolation	EC2 details for App-A and App-B	No public IPv4	Fill after check
Controlled failure	Remove App-B inbound ICMP rule temporarily; ping fails; restoring rule restores replies	Fill after test	Fill after test

Attach labelled screenshots in the evidence section or appendices. Do not represent expected outcomes as completed tests.

8. Troubleshooting

• SSH timeout to bastion: verify its current public IPv4, your current public IP /32 rule, public route-table association, and Internet Gateway route.

• SSH failure to a private instance: verify the key, ec2-user username, host private IP, bastion /32 security rule, and both peering routes for App-B.

• Ping failure: confirm the peering state is Active, both directions have routes, the destination security group allows ICMP Echo Request, and the private IPs match.

• Peering request failure: check all VPC CIDRs, including secondary CIDRs, for overlap.

9. Cost, cleanup and security

EC2 compute, EBS storage, and public IPv4 resources may incur charges. Check current AWS pricing and any account credits before deployment. After capturing evidence, terminate the three lab instances, check and delete leftover volumes or snapshots, release any Elastic IP, delete the peering connection and its routes, then remove lab security groups, subnets, custom route tables, the Internet Gateway, and both VPCs. Check Billing for remaining resources. Never upload private keys, passwords, or access tokens to GitHub.

10. Learning outcomes

On completion, the student should be able to explain non-overlapping CIDRs, local and peering routes, why return routes are necessary, how stateful security groups filter traffic, why VPC peering does not automatically provide connectivity or internet access, and how to diagnose a controlled connectivity failure.

11. Conclusion

The project provides a practical demonstration of AWS VPC networking, secure administration through a bastion host, and private workload communication across peered VPCs. Final conclusions should be updated after the student completes the deployment and records actual evidence.

Appendix A — Resource inventory

Record the actual AWS Region and resource IDs here: VPC A ID: _____; VPC B ID: _____; peering ID: _____; bastion instance ID/public IPv4: _____; App-A instance ID: _____; App-B instance ID: _____.

Appendix B — Evidence checklist

- ■ Architecture and actual resource inventory
- ■ Subnet CIDRs and three route tables with subnet associations
- ■ Active peering connection and inbound security-group rules
- ■ Baseline failure and forward/reverse ping success
- ■ SSH to App-B through the bastion
- ■ Controlled ICMP failure and recovery
- ■ Cleanup evidence